**Task 7: Gaussian Filtering for Smoothing**

Utilizes a 2D Gaussian kernel to suppress high-frequency spatial details and mitigate random image noise.

## 1. এই প্রোগ্রামটির মূল Concept কী?

এই প্রোগ্রামটির মূল কাজ হলো একটি RGB Image-এর Noise কমানো এবং Image Smooth করা Gaussian Filtering Technique ব্যবহার করে।

Gaussian Filtering হলো একটি Image Processing Technique, যার মাধ্যমে ছবির ছোট ছোট Noise এবং Unwanted Details কমিয়ে ছবিকে Smooth বা Blur করা হয়।

সহজভাবে বললে, একটি ছবিতে অনেক ছোট ছোট দাগ, Noise অথবা Unwanted Pixel Variation থাকলে Gaussian Filter ব্যবহার করে সেগুলো কমানো যায়।

তোমার দেওয়া Screenshot-এ দুটি Image দেখা যাচ্ছে:

1. Original Image: ছবির মূল অবস্থা, যেখানে Noise এবং ছোট ছোট Details রয়েছে।
2. Gaussian Filtered Image: Gaussian Filtering প্রয়োগ করার পর ছবিটি তুলনামূলক Smooth হয়েছে।

এখানে Original Image-এর প্রতিটি Pixel-এর আশেপাশের Pixel নিয়ে একটি Weighted Average Calculate করা হয়েছে। এরপর সেই Average Value দিয়ে নতুন Pixel তৈরি করা হয়েছে।

এইভাবে সম্পূর্ণ Image Process করে Gaussian Filtered Image তৈরি করা হয়েছে।

## 2. Gaussian Filtering কীভাবে কাজ করে?

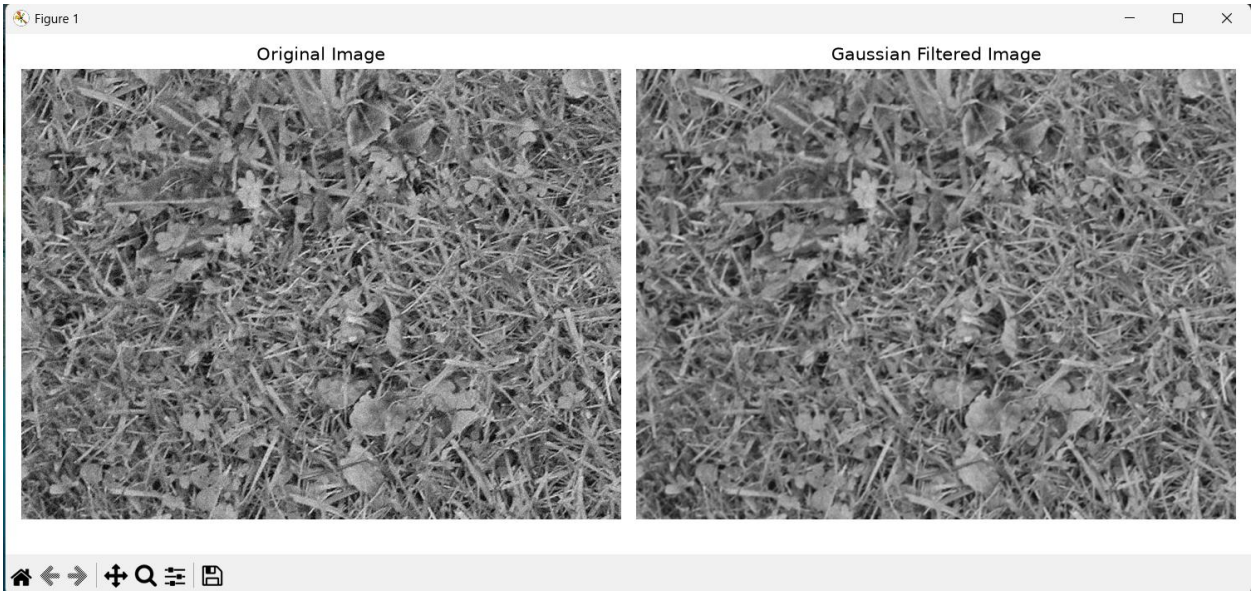
Gaussian Filtering মূলত একটি Weighted Average Technique।

এখানে প্রতিটি Pixel-এর আশেপাশের Pixel-গুলোকে বিভিন্ন Weight দেওয়া হয়।

- Center Pixel সবচেয়ে বেশি Weight পায়।
- Center-এর কাছাকাছি Pixel তুলনামূলক বেশি Weight পায়।
- দূরের Pixel তুলনামূলক কম Weight পায়।

অর্থাৎ, Center Pixel-এর আশেপাশের Pixel-এর তথ্য ব্যবহার করে নতুন Pixel Value Calculate করা হয়।

এই কাজের জন্য আমরা একটি Gaussian Kernel ব্যবহার করি।




তোমার প্রোগ্রামে:

kernel_size = 5 sigma = 1.0

এখানে:

- kernel_size = 5 অর্থ 5 × 5 Kernel ব্যবহার করা হচ্ছে।
- sigma = 1.0 অর্থ Gaussian Distribution-এর Standard Deviation 1.0।

অর্থাৎ, প্রতিটি Pixel Process করার সময় তার আশেপাশের 5 × 5 এলাকার মোট 25টি Pixel বিবেচনা করা হবে।

**3. Mathematical Formula**

Gaussian Filtering-এর জন্য ব্যবহৃত 2D Gaussian Formula হলো:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

এখানে:

- $G(x,y)$ = Gaussian Kernel-এর Weight।
- $x$ = Horizontal Distance।
- $y$= Vertical Distance।
- $\sigma$= Standard Deviation।
- $\pi$ = Mathematical Constant।

এই Formula ব্যবহার করে Kernel-এর প্রতিটি Position-এর Weight Calculate করা হয়।

এরপর সব Weight-এর যোগফল দিয়ে প্রতিটি Weight-কে Divide করা হয়।

$$G_{normalized} = \frac{G}{\sum G}$$

এর ফলে Kernel-এর সব Weight-এর যোগফল 1 হয়।

Final Pixel Calculation:

$$NewPixel = \sum (Pixel \times KernelWeight)$$

অর্থাৎ, প্রতিটি Pixel-এর Value-এর সঙ্গে সংশ্লিষ্ট Kernel Weight Multiply করে সবগুলো যোগ করা হয়। এই যোগফলই নতুন Pixel Value।

**4. সম্পূর্ণ Code (বাংলা Comment সহ)**

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
import math


image = Image.open("PNG/Task_07_Gaussian_Filtering.png").convert("RGB")
image_array = np.array(image, dtype=np.float64)


height, width, channels = image_array.shape


kernel_size = 5
sigma = 1.0


radius = kernel_size // 2
```

```
kernel = np.zeros((kernel_size, kernel_size), dtype=np.float64)

for i in range(kernel_size):
    for j in range(kernel_size):

        x = j - radius
        y = i - radius

        value = (1 / (2 * math.pi * sigma ** 2)) * math.exp(
            -(x ** 2 + y ** 2) / (2 * sigma ** 2)
        )

        kernel[i, j] = value

kernel_sum = np.sum(kernel)
kernel = kernel / kernel_sum

print("Gaussian Kernel:")
print(np.round(kernel, 4))

filtered_image = np.zeros(
    (height, width, channels),
    dtype=np.uint8
)
```

```
for y in range(height):
    for x in range(width):

        for c in range(channels):

            pixel_sum = 0.0

            for ky in range(kernel_size):
                for kx in range(kernel_size):
                    input_y = y + ky - radius
                    input_x = x + kx - radius
                    if input_y < 0:
                        input_y = 0
                    elif input_y >= height:
                        input_y = height - 1

                    if input_x < 0:
                        input_x = 0
                    elif input_x >= width:
                        input_x = width - 1
                    pixel_value = image_array[input_y, input_x, c]
                    kernel_value = kernel[ky, kx]
```

```
                    pixel_sum += pixel_value * kernel_value
            pixel_sum = max(0, min(255, pixel_sum))

            filtered_image[y, x, c] = int(round(pixel_sum))

output_image = Image.fromarray(filtered_image)
output_image.save("gaussian_filtered.png")

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_array.astype(np.uint8))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(filtered_image)
plt.title("Gaussian Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Gaussian filtering completed successfully.")
```

```
print("Output saved as gaussian_filtered.png")
```

**5. Step-by-Step Code Explanation**

**Step 1: Import Libraries**

from PIL import Image import numpy as np import matplotlib.pyplot as plt import math

এখানে আমরা চারটি Library Import করেছি।

- PIL (Pillow) → Image Open এবং Save করার জন্য।
- NumPy → Array তৈরি এবং Mathematical Calculation করার জন্য।
- Matplotlib → Original এবং Filtered Image Display করার জন্য।
- Math → Gaussian Formula-তে Exponential এবং Pi ব্যবহার করার জন্য।

**Step 2: Open the Image**

image = Image.open("PNG/Task_07_Gaussian_Filtering.png").convert("RGB")

এই লাইনে নির্দিষ্ট Location থেকে Original Image Open করছি।

এরপর .convert("RGB") ব্যবহার করে ছবিটিকে RGB Format-এ Convert করছি।

এর ফলে প্রতিটি Pixel-এর মধ্যে Red, Green এবং Blue Channel থাকবে।

**Step 3: Convert Image into NumPy Array**

image_array = np.array(image, dtype=np.float64)

এখানে Original Image-কে NumPy Array-তে Convert করছি।

কেন NumPy Array ব্যবহার করছি?

কারণ Gaussian Filtering-এর সময় অনেকগুলো Mathematical Calculation করতে হবে। NumPy Array ব্যবহার করলে Pixel Data নিয়ে সহজে Calculation করা যায়।

এখানে dtype=np.float64 ব্যবহার করেছি, যাতে Decimal Value নিয়েও Calculation করা যায়।

**Step 4: Get Image Dimensions**

height, width, channels = image_array.shape

এখানে ছবির তিনটি Dimension বের করছি।

- height = ছবির Vertical Pixel সংখ্যা।
- width = ছবির Horizontal Pixel সংখ্যা।

- channels = Color Channel সংখ্যা।

যেহেতু Image RGB, তাই channels = 3 হবে।

**Step 5: Define Gaussian Parameters**

kernel_size = 5 sigma = 1.0 radius = kernel_size // 2

এখানে Gaussian Filter-এর প্রয়োজনীয় Parameter নির্ধারণ করছি।

kernel_size = 5 অর্থ আমরা 5 × 5 Kernel ব্যবহার করছি।

sigma = 1.0 হলো Gaussian Distribution-এর Standard Deviation।

এরপর:

radius = kernel_size // 2

এখানে Integer Division ব্যবহার করে Radius বের করছি।

যেহেতু Kernel Size 5:

$$Radius = 5//2 = 2$$

অর্থাৎ Kernel-এর Center থেকে দুই Pixel বাম, ডান, উপরে এবং নিচে পর্যন্ত কাজ করবে।

**Step 6: Create an Empty Gaussian Kernel**

kernel = np.zeros( (kernel_size, kernel_size), dtype=np.float64)

এখানে NumPy-এর zeros() Function ব্যবহার করে একটি 5 × 5 Empty Kernel তৈরি করছি।

প্রথমে Kernel-এর প্রতিটি Position-এ Zero থাকবে।

পরবর্তীতে Gaussian Formula ব্যবহার করে প্রতিটি Position-এর Weight বসানো হবে।

**Step 7: Calculate Gaussian Kernel Values**

for i in range(kernel_size): for j in range(kernel_size): x = j - radius y = i - radius

এখানে দুটি Nested Loop ব্যবহার করছি।

এই Loop-এর মাধ্যমে Kernel-এর প্রতিটি Position Access করছি।

x এবং y ব্যবহার করে Kernel-এর Center থেকে প্রতিটি Position-এর Distance বের করছি।

এরপর:

value = (1 / (2 * math.pi * sigma ** 2)) * math.exp( -(x ** 2 + y ** 2) / (2 * sigma ** 2))

এখানে Gaussian Mathematical Formula ব্যবহার করে প্রতিটি Position-এর Weight Calculate করছি।

এরপর:

kernel[i, j] = value

Calculated Weight-টি Kernel-এর নির্দিষ্ট Position-এ সংরক্ষণ করছি।

এভাবে 5 × 5 Kernel-এর সবগুলো Weight Calculate করা হচ্ছে।

**Step 8: Normalize the Gaussian Kernel**

kernel_sum = np.sum(kernel)kernel = kernel / kernel_sum

এটি একটি গুরুত্বপূর্ণ ধাপ।

প্রথমে np.sum(kernel) ব্যবহার করে Kernel-এর সব Weight যোগ করছি।

এরপর প্রতিটি Weight-কে Total Sum দিয়ে Divide করছি।

একে Kernel Normalization বলা হয়।

এর ফলে Kernel-এর সব Weight-এর যোগফল 1 হয়।

কেন Normalization প্রয়োজন?

যাতে Filtering করার সময় ছবির Overall Brightness অপ্রয়োজনীয়ভাবে পরিবর্তিত না হয়।

**Step 9: Create an Empty Output Image**

filtered_image = np.zeros(  (height, width, channels),  dtype=np.uint8)

এখানে Original Image-এর সমান Dimension ব্যবহার করে একটি নতুন Empty Output Image তৈরি করছি।

এখানে:

- Height = Original Image-এর Height।
- Width = Original Image-এর Width।
- Channels = 3 (RGB)।

np.uint8 ব্যবহার করেছি, কারণ প্রতিটি Pixel Channel-এর Value 0 থেকে 255-এর মধ্যে Integer হিসেবে সংরক্ষণ করতে হবে।

**Step 10: Apply Gaussian Filtering**

for y in range(height):  for x in range(width):  for c in range(channels):  pixel_sum = 0.0

এখানে তিনটি Nested Loop ব্যবহার করেছি।

প্রথম Loop ছবির প্রতিটি Row Access করে।

দ্বিতীয় Loop প্রতিটি Column Access করে।

তৃতীয় Loop RGB-এর তিনটি Channel-এর উপর কাজ করে।

অর্থাৎ:

- c = 0 → Red Channel।
- c = 1 → Green Channel।
- c = 2 → Blue Channel।

প্রতিটি Pixel এবং Channel-এর জন্য pixel_sum = 0.0 দিয়ে Calculation শুরু করছি।

**Step 11: Access Neighboring Pixels**

for ky in range(kernel_size):  for kx in range(kernel_size):  input_y = y + ky - radius input_x = x + kx - radius

এখানে আরও দুটি Nested Loop ব্যবহার করেছি।

এই Loop-এর মাধ্যমে বর্তমান Pixel-এর আশেপাশের 5 × 5 এলাকার Pixel Access করছি।

মোট 25টি Pixel বিবেচনা করা হচ্ছে।

এখানে:

- ky = Kernel-এর Row Position।
- kx = Kernel-এর Column Position।
- input_y = Original Image-এর Pixel Row।
- input_x = Original Image-এর Pixel Column।

অর্থাৎ Kernel-কে Original Image-এর প্রতিটি Pixel-এর উপর বসিয়ে তার আশেপাশের Pixel সংগ্রহ করছি।

**Step 12: Handle Image Boundaries**

if input_y < 0:  input_y = 0 elif input_y >= height:  input_y = height – 1

if input_x < 0:  input_x = 0 elif input_x >= width:  input_x = width - 1

এটি এই প্রোগ্রামের একটি গুরুত্বপূর্ণ অংশ।

Image-এর Border বা Edge-এ কাজ করার সময় Kernel-এর কিছু Position ছবির বাইরে চলে যেতে পারে।

যেমন, ছবির প্রথম Pixel-এর বাম পাশের কোনো Pixel Access করতে গেলে Negative Coordinate পাওয়া যেতে পারে।

এটি সমাধান করার জন্য এখানে Boundary Handling করা হয়েছে।

এই পদ্ধতিকে Border Replication বা Edge Clamping বলা হয়।

অর্থাৎ, ছবির বাইরে চলে গেলে সবচেয়ে কাছের Border Pixel-এর Coordinate ব্যবহার করা হচ্ছে।

**Step 13: Multiply Pixel with Kernel Weight**

pixel_value = image_array[input_y, input_x, c]

kernel_value = kernel[ky, kx]

pixel_sum += pixel_value * kernel_value

এটি Gaussian Filtering-এর সবচেয়ে গুরুত্বপূর্ণ অংশ।

প্রথমে Original Image থেকে Neighboring Pixel-এর Value সংগ্রহ করছি।

এরপর Gaussian Kernel থেকে সংশ্লিষ্ট Weight সংগ্রহ করছি।

তারপর Pixel Value-এর সঙ্গে Kernel Weight Multiply করছি।

সবগুলো Result pixel_sum-এর মধ্যে যোগ করছি।

এভাবে 5 × 5 Kernel-এর 25টি Pixel-এর Weighted Sum পাওয়া যাচ্ছে।

**Step 14: Limit Pixel Value**

pixel_sum = max(0, min(255, pixel_sum))

এখানে Pixel Value-কে 0 থেকে 255-এর মধ্যে সীমাবদ্ধ রাখছি।

কারণ RGB Image-এর প্রতিটি Channel-এর Value 0 থেকে 255-এর মধ্যে হতে হয়।

**Step 15: Update Output Pixel**

filtered_image[y, x, c] = int(round(pixel_sum))

এখানে Calculated Weighted Average-কে round() ব্যবহার করে পূর্ণসংখ্যায় Convert করছি।

এরপর নতুন Output Image-এর একই Coordinate এবং Channel-এ Value বসিয়ে দিচ্ছি।

এভাবে প্রতিটি Pixel-এর জন্য নতুন Value তৈরি হচ্ছে।

**Step 16: Convert Array into Image**

output_image = Image.fromarray(filtered_image)

এখানে NumPy Array-কে আবার PIL Image-এ Convert করছি।

কারণ পরবর্তীতে Image-টি Save করতে হবে।

**Step 17: Save Filtered Image**

output_image.save("gaussian_filtered.png")

এখানে Gaussian Filtering সম্পন্ন হওয়া Image-টি gaussian_filtered.png নামে Save করছি।

**Step 18: Display Original and Filtered Images**

plt.figure(figsize=(12, 5))

এখানে Matplotlib-এর মাধ্যমে 12 × 5 Size-এর Figure তৈরি করছি।

এরপর:

plt.subplot(1, 2, 1)

plt.imshow(image_array.astype(np.uint8))

plt.title("Original Image")plt.axis("off")

এখানে প্রথম Position-এ Original Image Display করছি।

এরপর:

plt.subplot(1, 2, 2)

plt.imshow(filtered_image)

plt.title("Gaussian Filtered Image")plt.axis("off")

দ্বিতীয় Position-এ Gaussian Filtering করা Image Display করছি।

সবশেষে:

plt.tight_layout()

plt.show()

tight_layout() ব্যবহার করে দুইটি Image-এর মধ্যকার Spacing ঠিক করছি।

show() ব্যবহার করে Final Result Display করছি।

**Oral Explanation (বাংলা)**

প্রথমে আমি PIL, NumPy, Matplotlib এবং Math Library Import করেছি।

এরপর একটি Original RGB Image Open করেছি এবং সেটিকে NumPy Array-তে Convert করেছি।

তারপর ছবির Height, Width এবং Color Channel সংখ্যা বের করেছি।

এরপর Gaussian Filter-এর জন্য Kernel Size 5 এবং Sigma Value 1.0 নির্ধারণ করেছি। তারপর একটি 5 × 5 Empty Kernel তৈরি করেছি।

এরপর দুটি Nested Loop ব্যবহার করে Gaussian Mathematical Formula প্রয়োগ করে Kernel-এর প্রতিটি Position-এর Weight Calculate করেছি।

সব Weight Calculate করার পর সেগুলো যোগ করেছি এবং প্রতিটি Weight-কে Total Sum দিয়ে Divide করেছি। এই প্রক্রিয়াকে Kernel Normalization বলা হয়।

এরপর Original Image-এর সমান Dimension ব্যবহার করে একটি নতুন Empty Output Image তৈরি করেছি।

তারপর তিনটি Nested Loop ব্যবহার করে ছবির প্রতিটি Pixel এবং RGB Channel Access করেছি।

প্রতিটি Pixel-এর জন্য আরও দুটি Nested Loop ব্যবহার করে তার আশেপাশের 5 × 5 এলাকার মোট 25টি Pixel সংগ্রহ করেছি।

এরপর প্রতিটি Pixel Value-এর সঙ্গে সংশ্লিষ্ট Gaussian Kernel Weight Multiply করে সবগুলো যোগ করেছি।

Image-এর Border-এ সমস্যা এড়ানোর জন্য Boundary Handling করেছি, যেটিকে Edge Clamping বলা হয়।

এরপর Calculated Pixel Value-কে 0 থেকে 255-এর মধ্যে সীমাবদ্ধ রেখে নতুন Output Image-এ বসিয়েছি।

সব Pixel Process করা শেষ হলে NumPy Array-কে PIL Image-এ Convert করেছি এবং Gaussian Filtered Image Save করেছি।

সবশেষে Original Image এবং Gaussian Filtered Image পাশাপাশি Display করেছি।

এভাবেই Gaussian Filtering ব্যবহার করে একটি Image Smooth করেছি এবং Noise কমিয়েছি।

**7. Viva Questions & Answers**

Q1. What is Gaussian Filtering?

Ans: Gaussian Filtering is a smoothing technique used to reduce image noise and blur unwanted details.

Q2. What is a Gaussian Kernel?

Ans: A Gaussian Kernel is a matrix of weights calculated using the Gaussian mathematical function.

Q3. What is the purpose of Sigma?

Ans: Sigma controls the spread of the Gaussian distribution and determines the amount of smoothing.

Q4. What is Kernel Size?

Ans: Kernel Size determines the size of the neighborhood used for filtering. For example, 5 × 5 means 25 neighboring pixels are considered.

Q5. What is Kernel Normalization?

Ans: Kernel Normalization is the process of dividing each kernel value by the sum of all kernel values so that their total becomes 1.

Q6. Why do we use NumPy?

Ans: NumPy is used for numerical calculations and efficient handling of image pixel data.

Q7. Why do we use Nested Loops?

Ans: To access every pixel, every color channel and all neighboring pixels.

Q8. What is Boundary Handling?

Ans: Boundary Handling is a technique used to process pixels near image edges without accessing invalid coordinates.

Q9. What is Edge Clamping?

Ans: Edge Clamping replaces out-of-range pixel coordinates with the nearest valid border coordinate.

Q10. Why do we use pixel_sum += pixel_value * kernel_value?

Ans: It calculates the weighted sum of neighboring pixel values using Gaussian kernel weights.

Q11. What is the purpose of np.zeros()?

Ans: It creates an array filled with zeros.

Q12. What is the purpose of np.sum()?

Ans: It calculates the sum of all elements in an array.

Q13. What happens when Sigma increases?

Ans: Generally, increasing Sigma produces stronger smoothing and more blurring.

Q14. Does Gaussian Filtering remove all noise?

Ans: No. It reduces noise but does not remove all types of noise completely.

Q15. What is the difference between Original Image and Gaussian Filtered Image?

Ans: The Original Image contains its original details and noise, whereas the Gaussian Filtered Image is smoother with reduced high-frequency variations.

## 8. গুরুত্বপূর্ণ Functions ও Variables মনে রাখবে

| **Code / Variable** | **কাজ** |
|---|---|
| Image.open() | Image Open করে |
| np.array() | Image-কে NumPy Array করে |
| image_array.shape | Height, Width, Channels বের করে |
| np.zeros() | Empty Array তৈরি করে |
| kernel_size | Kernel-এর Dimension নির্ধারণ করে |
| sigma | Gaussian Distribution-এর Spread নির্ধারণ করে |
| math.exp() | Exponential Calculation করে |
| math.pi | Pi Constant প্রদান করে |
| np.sum() | সব Weight যোগ করে |
| kernel / kernel_sum | Kernel Normalize করে |
| image_array[y,x,c] | নির্দিষ্ট Pixel Channel Access করে |
| pixel_sum | Weighted Sum সংরক্ষণ করে |
| np.uint8 | 8-bit Integer Data Type |

| | |
|---|---|
| Image.fromarray() | Array-কে Image-এ Convert করে |
| plt.imshow() | Image Display করে |
| plt.subplot() | একাধিক Image Display-এর Layout তৈরি করে |

## 9. পরীক্ষার আগে সবচেয়ে গুরুত্বপূর্ণ ৫টি বিষয়

1. Gaussian Kernel Creation

kernel_size = 5sigma = 1.0

5 × 5 Kernel এবং Sigma নির্ধারণ করে।

2. Gaussian Formula

value = (1 / (2 * math.pi * sigma ** 2)) * math.exp(  -(x ** 2 + y ** 2) / (2 * sigma ** 2))

Kernel-এর প্রতিটি Weight Calculate করে।

3. Kernel Normalization

kernel_sum = np.sum(kernel)kernel = kernel / kernel_sum

সব Weight-এর যোগফল 1 করে।

4. Weighted Sum

pixel_sum += pixel_value * kernel_value

Neighboring Pixel এবং Kernel Weight ব্যবহার করে নতুন Pixel Value Calculate করে।

5. Output Pixel Update

filtered_image[y, x, c] = int(round(pixel_sum))

নতুন Pixel Value Output Image-এ বসায়।

**Key Caveats:**

1. Gaussian Filtering ছবির Noise কমায়, তবে একই সঙ্গে কিছু Fine Details এবং Edge-ও Blur করতে পারে।
2. Kernel Size 5 এবং Sigma 1.0 এই প্রোগ্রামের নির্ধারিত Parameter; অন্য মান ব্যবহার করলে ফলাফল পরিবর্তিত হবে।
3. এখানে RGB-এর তিনটি Channel আলাদাভাবে Filter করা হয়েছে।
4. Border Pixel-এর জন্য Edge Clamping ব্যবহার করা হয়েছে।

5. এই কোডে Manual Nested Loop ব্যবহার করা হয়েছে, তাই বড় Image-এর ক্ষেত্রে Processing Time বেশি হতে পারে।

Exam Tip: এই প্রোগ্রামের সবচেয়ে গুরুত্বপূর্ণ Concept হলো:

Gaussian Formula → Kernel Creation → Kernel Normalization → Weighted Sum → Pixel Update।